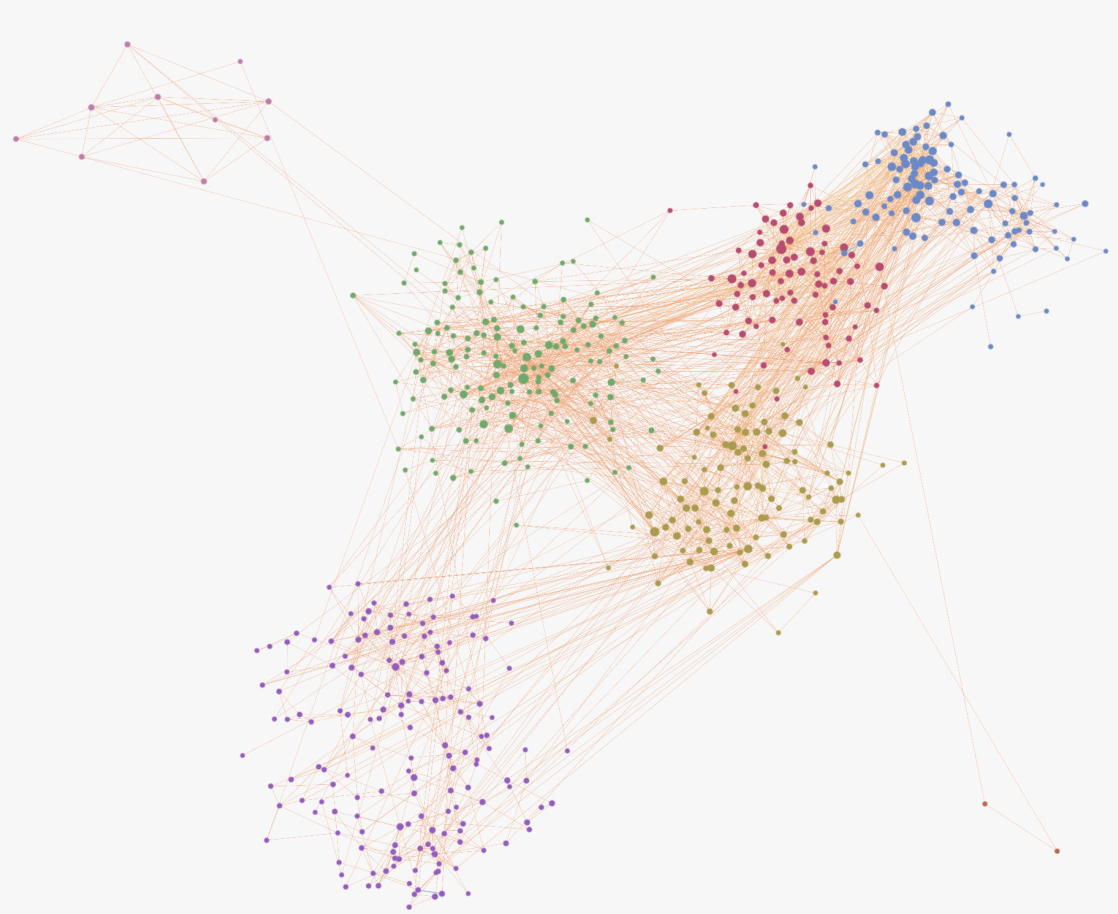
MODAL Graph Analysis Project

Modal Graph Analysis

Repository and Company Community Networks



Raphael des Boscs Xufan Huang

April 2026

Contents

1 Abstract **2**

2 Research Questions **2**

3 Data Acquisition and Cleaning **2**
3.1 Sampling Strategy . . . 2
3.2 Two-step GraphQL Retrieval . . . 2
3.3 GitHub GraphQL: Role in Our Study and Queries . . . 3
3.4 Raw Fields Collected . . . 4
3.5 Cleaning Rules . . . 4

4 Repository-level Graph Construction **5**
4.1 Method Evolution (from Topics to Hybrid Similarity) . . . 5
4.2 Node Definition . . . 5
4.3 Edge Weight Design . . . 5

5 Backbone Extraction and Denoising **6**
5.1 Auto-relax Strategy . . . 6

6 Community Detection and Centrality **6**
6.1 Louvain Community Detection . . . 6
6.2 Centrality Metrics . . . 6

7 Company-level Aggregation **7**
7.1 Mapping Owner to Company . . . 7
7.2 Company Graph . . . 7

8 Results by Sampling Stage **7**
8.1 Stage A: Initial Repository-level Analysis (100+100) . . . 7
8.2 Signal Ablation Comparison: Fork-only vs README-only vs Mixed . . . 7
8.3 Stage B: Expanded Run for Company Analysis (500+500) . . . 9
8.4 Company Graph Visualization (Stage B) . . . 10
8.5 Interpretation of the Five Largest Company Communities (Stage B) . . . 10
8.6 Hypothesis Test: Are Large Companies More Central? . . . 12

9 Visualization Protocol **13**
9.1 Why "hairball" happens . . . 13
9.2 Recommended Gephi Layout Template . . . 13

10 Threats to Validity **14**

11 Reproducibility **14**
11.1 Environment . . . 14
11.2 Main Commands . . . 14

12 Conclusion **15**

1 Abstract

This report presents a multi-layer network analysis pipeline for AI repositories on GitHub. Starting from topic-seeded repository retrieval (`computer-vision` and `nlp`), we construct a repository-level weighted graph, extract backbone structure, detect communities, and compute centrality metrics. We then aggregate the repository graph to a company-level graph to evaluate whether ownership entities form distinct communities and whether small entities cluster together. Methodologically, the project follows an iterative path: an initial topic-centric attempt on top100+top100 repositories, then single-signal experiments (fork-overlap only and README-similarity only), then a mixed-weight graph with community/centrality analysis, and finally company-level aggregation. The report focuses on methodological design, explicit mathematical definitions, and reproducible implementation details.

2 Research Questions

We focus on the following questions:

1. **RQ1**: Can a repository-level network built from fork, README, and dependency signals reveal interpretable AI communities at scale?
2. **RQ2**: At company level, do ownership entities form distinct communities, and are these communities dominated by small entities?
3. **RQ3**: Which repositories and companies play bridge roles across communities?

3 Data Acquisition and Cleaning

3.1 Sampling Strategy

We use two query seeds on GitHub GraphQL:

- `topic:computer-vision`
- `topic:nlp`

Our experiments were conducted in two stages:

- **Stage A (initial repository-level run):** top **100+100** repositories (CV+NLP, about 200 raw repositories before deduplication), used to validate the repository-level graph pipeline and obtain the first interpretable repo communities.
- **Stage B (expanded company-level run):** because company communities from the 100+100 setting were too small/fragmented for stable interpretation, we expanded to top **500+500** repositories and reran the full pipeline for company-level analysis.

In Stage B, after cross-domain deduplication, the unique repository set contains **967 repositories**.

3.2 Two-step GraphQL Retrieval

To reduce request instability and payload size:

1. **List stage**: retrieve compact repository metadata (`nameWithOwner`, stars, forks, language).
2. **Detail stage**: per repository, retrieve topics, README text, dependency files, and forks.

3.3 GitHub GraphQL: Role in Our Study and Queries

Brief introduction GitHub GraphQL is a typed query API where we request exactly the fields needed for each repository. Compared with broad REST crawling, this reduces payload redundancy and enables nested retrieval (repository metadata, topics, README blobs, dependencies, forks) in a controlled schema.

Why it matters for this project It is the data backbone of our pipeline:

- supports **topic-seeded sampling** for CV/NLP repositories;
- enables **two-step retrieval** (light list pass, then deep detail pass);
- provides the raw signals used in edge construction (`fork overlap`, `README similarity`, `dependency overlap`).

Search query string used in the list stage For each domain, we query:

`topic:{domain} sort:stars-desc`

where `domain` is `computer-vision` or `nlp`.

GraphQL query used in the list stage (`LIST_REPOS_QUERY`)

```
query($searchQuery: String!, $cursor: String, $first: Int!) {
  search(query: $searchQuery, type: REPOSITORY, first: $first, after: $cursor) {
    pageInfo { hasNextPage endCursor }
    nodes {
      ... on Repository {
        nameWithOwner
        stargazerCount
        forkCount
        primaryLanguage { name }
      }
    }
  }
}
```

GraphQL query used in the detail stage (`DETAIL_QUERY`)

```
query(
  $owner: String!,
  $name: String!,
  $topicsFirst: Int!,
  $forksFirst: Int!
) {
  repository(owner: $owner, name: $name) {
    nameWithOwner
    stargazerCount
    forkCount
    isFork
    primaryLanguage { name }
    parent { nameWithOwner }
    repositoryTopics(first: $topicsFirst) {
```

```
      nodes { topic { name } }
    }
    readmeMd: object(expression: "HEAD:README.md") { ... on Blob { text } }
    readmeLower: object(expression: "HEAD:readme.md") { ... on Blob { text } }
    readmeRst: object(expression: "HEAD:README.rst") { ... on Blob { text } }
    readmeTxt: object(expression: "HEAD:README.txt") { ... on Blob { text } }
    readme: object(expression: "HEAD:README") { ... on Blob { text } }

    pkgJson: object(expression: "HEAD:package.json") { ... on Blob { text } }
    reqTxt: object(expression: "HEAD:requirements.txt") { ... on Blob { text } }
    pyprojectToml: object(expression: "HEAD:pyproject.toml") { ... on Blob { text } }
    goMod: object(expression: "HEAD:go.mod") { ... on Blob { text } }
    cargoToml: object(expression: "HEAD:Cargo.toml") { ... on Blob { text } }
    pomXml: object(expression: "HEAD:pom.xml") { ... on Blob { text } }
    gemfile: object(expression: "HEAD:Gemfile") { ... on Blob { text } }
    envYml: object(expression: "HEAD:environment.yml") { ... on Blob { text } }
    forks(first: $forksFirst) {
      totalCount
      nodes {
        nameWithOwner
        isArchived
        pushedAt
        stargazerCount
        owner { login }
      }
    }
  }
}
```

3.4 Raw Fields Collected

Per repository, we collect:

- Basic: `nameWithOwner`, `stargazerCount`, `forkCount`, `primaryLanguage`
- Content: README text (multiple path fallbacks)
- Dependencies: `requirements.txt`, `pyproject.toml`, `package.json`, `go.mod`, `Cargo.toml`, `pom.xml`, `Gemfile`, `environment.yml`
- Social signal: fork owners
- Domain context: source topic-domain assignment (CV / NLP / cross-domain)

3.5 Cleaning Rules

User/account filtering:

- Remove any login containing `[bot]`
- Remove `web-flow`

Dependency names and company labels are normalized to reduce lexical duplication.

4 Repository-level Graph Construction

4.1 Method Evolution (from Topics to Hybrid Similarity)

We did **not** start directly from the final mixed-weight graph. Instead, the repository-level design evolved in the following order:

1. **Topic-centric trial on top100+top100 raw data**: we first attempted to organize repositories mainly by topic metadata.
2. **Observation**: topic fields were noisy in practice (missing tags, inconsistent granularity, and heterogeneous naming), which made pure topic grouping unstable.
3. **Signal shift**: we moved to structural/textual similarity, prioritizing fork-overlap and README semantic similarity.
4. **Single-signal ablations**: we generated separate views with fork-overlap only and README-similarity only, then compared their strengths and limitations.
5. **Final repository graph**: we adopted the hybrid weighting design and then applied backbone extraction + Louvain + centrality metrics.

This evolution explains why topics are kept as attributes/context, but not used as the only relation signal.

4.2 Node Definition

Each node is a repository. Topics are retained as node attributes (not topic nodes), avoiding excessive topic-projection density.

4.3 Edge Weight Design

For repositories i, j, edge weight is defined as:

$$w_{ij} = \alpha s_{ij}^{\text{fork}} + \beta s_{ij}^{\text{readme}} + \gamma s_{ij}^{\text{dep}} \tag{1}$$

with default parameters:

$$(\alpha, \beta, \gamma) = (0.6, 0.2, 0.2) \tag{2}$$

Fork-overlap signal

$$s_{ij}^{\text{fork}} = \frac{|F_i \cap F_j|}{|F_i \cup F_j|} \tag{3}$$

where F_i is the set of fork-owner accounts of repository i.

README semantic signal README text is preprocessed (remove code blocks, markdown links/images, low-information terms), then converted into sparse TF-IDF vectors which is in dictionary form by representing every word with the TF-IDF score. For term t in document d:

$$\text{tfidf}(t, d) = (1 + \log \text{tf}(t, d)) \left(\log \frac{1 + N}{1 + \text{df}(t)} + 1 \right) \tag{4}$$

README similarity:

$$s_{ij}^{\text{readme}} = \frac{v_i \cdot v_j}{\|v_i\| \|v_j\|} \tag{5}$$

Dependency-overlap signal

$$s_{ij}^{\text{dep}} = \frac{|D_i \cap D_j|}{|D_i \cup D_j|} \tag{6}$$

where D_i is the parsed dependency set.

5 Backbone Extraction and Denoising

The raw similarity graph is filtered in stages:

1. **Pre-threshold** (`pre-min-weight`) before edge insertion.
2. **Backbone threshold** (`min-weight`) on existing edges.
3. **Optional top-k sparsification** per node.
4. **Optional k-core pruning**.

5.1 Auto-relax Strategy

To avoid graph collapse under strict thresholds, we evaluate multiple relaxed candidate configurations and select by a size–modularity utility. This increases robustness for large and heterogeneous samples.

6 Community Detection and Centrality

6.1 Louvain Community Detection

Weighted Louvain is applied to the final backbone graph. Node attribute `Community` stores cluster IDs. Louvain greedily optimizes modularity:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \mathbf{1}(c_i = c_j) \tag{7}$$

where A_{ij} is weighted adjacency, k_i is weighted degree, and m is total edge weight. Intuitively, Louvain prefers partitions with denser-than-random intra-community connectivity.

6.2 Centrality Metrics

We compute:

- Weighted PageRank
- Weighted betweenness centrality (edge distance $= 1/w$)
- Weighted eigenvector centrality

Interpretation used in this report:

- **PageRank**: influence under recursive endorsement from important neighbors.
- **Betweenness centrality**: bridge score based on shortest paths crossing a node. For node v,

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \tag{8}$$

  where σ_{st} is the number of shortest paths from s to t, and $\sigma_{st}(v)$ counts those passing through v.

- **Eigenvector centrality**: importance by being connected to other important nodes.

In practice, high-betweenness repositories/companies are interpreted as cross-community connectors. These metrics are exported to GEXF for visualization and downstream analysis.

7 Company-level Aggregation

7.1 Mapping Owner to Company

Company entities are inferred from repository owner:

1. If owner is an **Organization**, we use organization name/login as company identity.
2. If owner is a **User**, we use the profile `company` field.
3. Owners with no company information are removed for the final company graph.

7.2 Company Graph

Company node features include: `RepoCount`, `TotalStars`, `OwnerCount`, `SmallCompany` flag.

For company pair (a, b), we aggregate repository edges and normalize:

$$W_{ab} = \frac{\sum_{(i,j)\in\mathcal{P}_{ab}} w_{ij}}{\sqrt{n_a n_b}} \tag{9}$$

where $\mathcal{P}_{ab}$ are cross-company repository pairs and n_a, n_b are repository counts per company.

Louvain is then applied again on the company graph, producing `CompanyCommunity`.

Weight semantics For each company-company edge, we keep both:

- `raw_weight`: direct sum of underlying repository-pair similarities;
- `weight`: normalized weight used in clustering and layout (W_{ab} above).

8 Results by Sampling Stage

8.1 Stage A: Initial Repository-level Analysis (100+100)

Our repository-level exploration started from the 100+100 raw setting. This stage was used to verify that the weighted repository graph and Louvain communities were meaningful, and to inspect bridge repositories before scaling. However, when projected to company level, the resulting company communities were too small for robust interpretation. Therefore, we kept Stage A for methodological validation and moved final company interpretation to Stage B.

8.2 Signal Ablation Comparison: Fork-only vs README-only vs Mixed

To isolate the contribution of each similarity signal, we compared three repository graphs: `fork-overlap only`, `README-similarity only`, and `mixed-weight`. This comparison is run only on the Stage A 100+100 repository data (`repo_raw_data_fork_top100.json`, 186 repositories after deduplication). All three GEXF outputs reuse the mixed-weight graph layout: the mixed graph is first clustered by its Louvain `Community`, then fork-only and README-only reuse the same coordinates for shared nodes. For this diagnostic visualization, the fork-only graph uses Jaccard threshold ≥ 0.0125, and the README-only graph uses cosine similarity ≥ 0.07. The generated files are `outputs/graphs/repo_signal_compare/repository_mixed_weight.gexf`, `repository_fork_only.gexf`, and `repository_readme_only.gexf`.

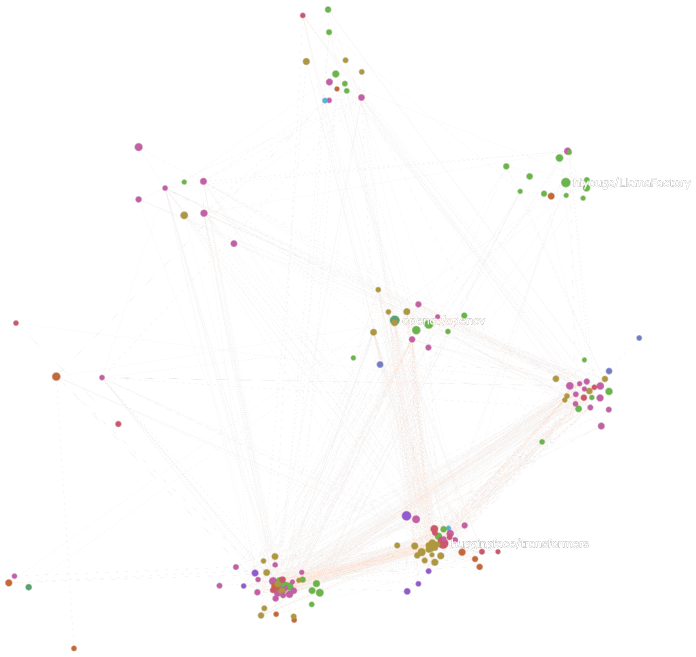


Figure 1: Fork-overlap-only repository graph on the Stage A 100+100 dataset. Nodes keep the mixed-weight layout coordinates and are colored by the mixed-weight community reference, making it possible to see which mixed communities are supported by shared fork audiences.

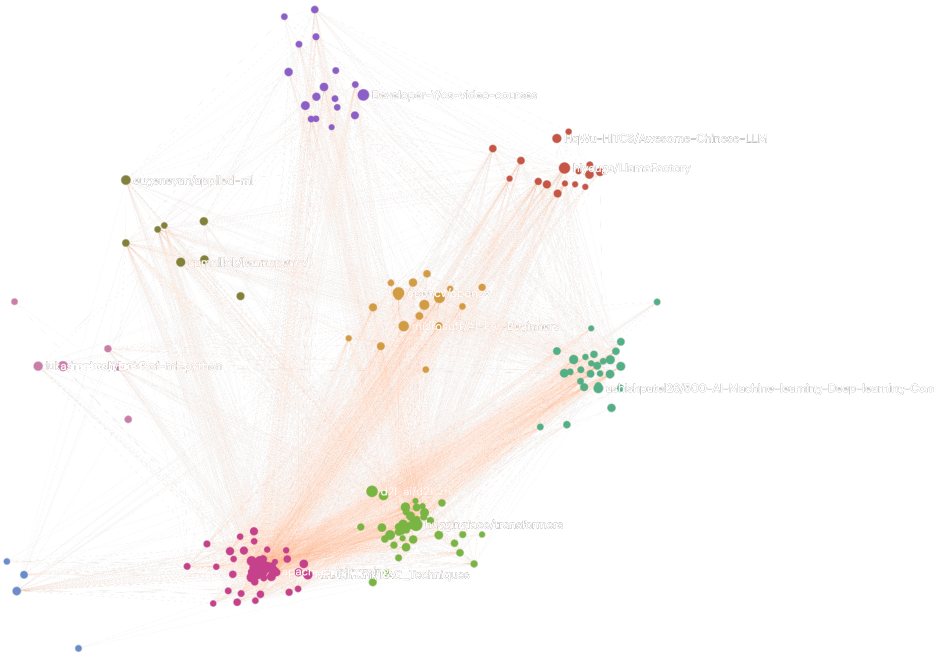


Figure 2: README-similarity-only repository graph on the Stage A 100+100 dataset. The same node coordinates and color reference are used as in Figure 1. Compared with fork-overlap, README similarity creates many more cross-community edges because general AI/ML README vocabulary overlaps broadly across repositories.

Table 1: Signal ablation summary (fixed-layout comparison)

Graph	Nodes	Edges	Communities	LCC	Density	Avg. degree	Modularity
Fork-only	160	872	9	160	0.0686	10.90	0.3974
README-only	177	4,580	5	177	0.2940	51.75	0.1532
Mixed-weight	153	1,178	9	149	0.1013	15.40	0.2708

Key differences Fork-only produces the clearest modular separation, because shared fork audiences are sparse and selective. README-only keeps broader coverage but is much denser, so communities are visually more compressed and modularity is lower. Mixed-weight sits between the two: it preserves enough bridges for global continuity while avoiding the excessive density of README-only.

The large density gap does **not** mean that denoising was absent from the README-only graph. Rather, the same denoising operation is applied to different edge-weight definitions. In README-only mode, the edge weight is the raw cosine similarity:

$$w_{ij} = s_{ij}^{\text{readme}}.$$

In mixed mode, the edge weight is:

$$w_{ij} = 0.6s_{ij}^{\text{fork}} + 0.2s_{ij}^{\text{readme}} + 0.2s_{ij}^{\text{dep}}.$$

Therefore, a threshold value has different meaning across the two graphs. For example, if a mixed-weight edge were supported only by README similarity, passing the mixed threshold $w_{ij} \geq 0.03$ would require:

$$0.2s_{ij}^{\text{readme}} \geq 0.03 \quad \Rightarrow \quad s_{ij}^{\text{readme}} \geq 0.15.$$

By contrast, the README-only visualization above uses $s_{ij}^{\text{readme}} \geq 0.07$, which is much more permissive. Empirically, with the Stage A data, README-only keeps 4,580 edges at threshold 0.07; raising the README threshold to about 0.106 would reduce it to roughly the same edge count as the mixed graph. This explains why README-only is visually much denser even though it is also threshold-filtered. README text captures broad semantic proximity, so repositories sharing generic AI terms such as "LLM", "transformer", "training", "dataset", "inference", or "model" can still be linked. Fork overlap is more selective because it requires common fork owners, which usually indicates a more specific shared audience or usage context.

8.3 Stage B: Expanded Run for Company Analysis (500+500)

Intermediate repository graph used for company aggregation The expanded 500+500 repository graph is used only as the substrate for company-level aggregation. It is not used for the repository-level signal ablation above.

How company nodes are constructed Each node is a company entity derived from repository ownership after filtering:

- organization owners are mapped directly to organization-level company IDs;
- user owners are mapped via profile `company`;
- owners without company information are excluded.

Thus, the node set emphasizes identifiable organizations rather than anonymous personal accounts.

Table 2: Intermediate repository graph summary for Stage B company aggregation (CV500 + NLP500)

Metric	Value
Unique repositories (after merge)	967
Backbone nodes	847
Backbone edges	22,893
Density	0.063897
Average path length (LCC)	2.795839
Communities	19
Modularity	0.2576

How company edges are constructed For companies a, b, we aggregate all repository-level edges crossing the two companies:

$$\text{raw_weight}_{ab} = \sum_{(i,j)\in\mathcal{P}_{ab}} w_{ij} \tag{10}$$

then normalize:

$$W_{ab} = \frac{\text{raw_weight}_{ab}}{\sqrt{n_a n_b}} \tag{11}$$

where n_a, n_b are repository counts of the two companies. This prevents large entities from dominating simply due to scale. In implementation, we keep an edge (a, b) only when at least one cross-company repository pair contributes non-zero similarity; self-loops are removed. Therefore, a company-company link means "evidence of technical relatedness" (fork overlap, README similarity, dependency overlap), not merely co-popularity.

Table 3: Company-level graph summary (owners without company removed)

Metric	Value
Company nodes	451
Company edges	9,010
Company communities	36
Modularity	0.2210

Company-level summary in Stage B (company-filtered run)

8.4 Company Graph Visualization (Stage B)

8.5 Interpretation of the Five Largest Company Communities (Stage B)

Current evidence indicates that the ownership graph has a heavy-tail structure: many small entities and a few high-connectivity anchors. The five largest company communities contain most nodes and should be prioritized for interpretation; tiny communities (size 1–3) are unstable and should not be over-interpreted.

Community meaning and examples The following interpretation is derived from `company_network_clust` (451 nodes, 9010 edges), not only from visual proximity. At the meso scale, communities `C0`, `C1`, `C3`, and `C4` form the structural core, while `C2` and most tiny communities behave as satellites.

Core-vs-satellite evidence. The six strongest inter-community channels are `C0-C4` (50.58), `C1-C4` (36.44), `C1-C3` (27.60), `C0-C3` (27.59), `C3-C4` (26.04), and `C0-C1` (23.61). Together they

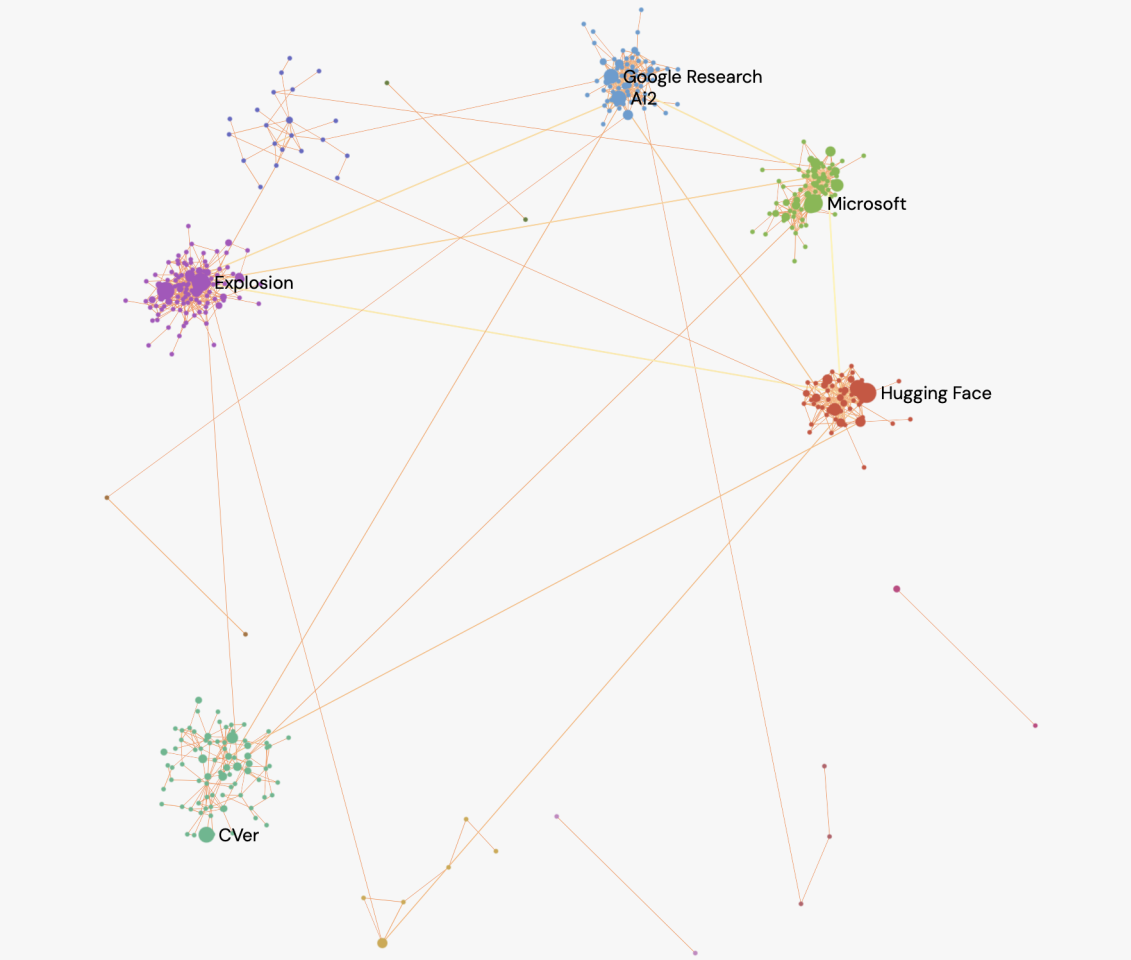


Figure 3: Final company-level community graph used in our final analysis. Nodes are companies, node size reflects activity/importance, and node color indicates `CompanyCommunity`.

Table 4: Five largest company communities (updated run)

Community ID	Company count	Small-company ratio	Median stars
3	114	0.9649	2415.0
2	81	0.9753	2040.0
0	74	0.9595	1679.5
4	70	0.9286	1687.5
1	50	0.9000	2122.5

account for about 97% of total cross-community edge weight. In contrast, all links involving `C2` are much weaker (`C2-C4`=1.36, `C0-C2`=1.21, `C2-C3`=0.95, `C1-C2`=0.94), which quantitatively supports the "peripheral CV enclave" interpretation for `C2`.

Community-level semantics with composition evidence.

- **C3 (114 nodes, dense applied NLP/CV tooling mix):** top repository owners include *Explosion* (12 repos), *Roboflow* (11), and *Google* (4). High-betweenness nodes (*Explosion*, *DeepPavlov Team*) indicate this community is a major transit layer between productized tooling and other ecosystems.
- **C0 (74 nodes, research-platform transfer hub):** anchored by *Ai2* (8), *Google Research* (8), and *PaddlePaddle* (4). Its strongest external coupling is toward `C4` and `C3`, suggesting a role as an upstream research source that diffuses methods into engineering communities.
- **C4 (70 nodes, industrial CV/infra engineering block):** anchored by *Microsoft* (12), *NVIDIA* (6), and *Alibaba* (4). Strong coupling with `C0` and `C1` reflects continuous exchange between industrial deployment stacks and research/NLP model ecosystems.
- **C1 (50 nodes, model-hub and language-centric block):** centered on *Hugging Face* (14), *Tencent* (10), *THUNLP* (6), and *Bytedance Inc.* (4). It is internally cohesive and strongly connected to `C4` and `C3`, consistent with foundation-model tooling and multilingual NLP transfer.
- **C2 (81 nodes, sparse and specialized CV-oriented satellite):** although large in node count (*CVer* and *DAIR.AI* are prominent), it has low internal density and weak links to all core communities, indicating narrower technical overlap after normalization.

Concrete bridge examples. High-weight cross-community edges include *@cohere-ai (C1)–Explosion (C3)* (0.1448), *Autodistill (C0)–Roboflow (C3)* (0.1274), *Ai2 (C0)–DeepPavlov Team (C3)* (0.1120), and *Explosion (C3)–@huggingface (C1)* (0.1126). These bridges explain why the four-core-community block remains strongly integrated despite clear color-separated modules in Figure 3.

8.6 Hypothesis Test: Are Large Companies More Central?

After describing the company communities, we test whether company size is associated with centrality. The hypothesis is:

$$H_0 : \ C_{\text{large}} \leq C_{\text{small}} \quad \text{vs.} \quad H_1 : \ C_{\text{large}} > C_{\text{small}},$$

where C is a company-level centrality score. Company size is defined from the sampled repository ownership data: **small companies** have `repo_count` ≤ 3, and **large companies** have `repo_count` > 3. This gives 21 large companies and 430 small companies in the company-filtered Stage B graph.

Because centrality distributions are highly skewed and the two groups are imbalanced, we use a one-sided Mann–Whitney U test rather than relying on a normality-based mean comparison. The tested metrics are graph degree, weighted degree (strength), PageRank, betweenness centrality, and eigenvector centrality. For multiple testing, we report Bonferroni-adjusted p-values across the five metrics. Effect size is reported as rank-biserial correlation, where positive values mean that large companies tend to have higher centrality ranks than small companies.

Table 5: One-sided Mann–Whitney U test for large-company centrality advantage

Metric	Large median	Small median	U	Adj. p	Effect
Degree	72.0000	12.0000	6195.0	0.0099	0.3721
Weighted degree	2.6491	0.3994	6057.5	0.0205	0.3416
PageRank	0.0031	0.0015	6028.5	0.0237	0.3352
Betweenness	0.0049	0.0002	6502.0	0.0014	0.4401
Eigenvector	0.0538	0.0088	5898.5	0.0443	0.3064

The results reject H_0 at $\alpha = 0.05$ for all five centrality metrics after Bonferroni correction. The strongest effect appears for betweenness centrality, suggesting that large companies are especially likely to act as bridges between company communities. This is consistent with the visual graph: large entities such as *Hugging Face*, *Microsoft*, *Explosion*, *Google Research*, and *Ai2* are not only locally connected inside their own communities, but also occupy routes between different technical ecosystems.

This result should be interpreted as an association, not a causal claim. The company graph already normalizes edge weights by $\sqrt{n_a n_b}$, so the result is not simply a mechanical consequence of large companies having more repositories. However, larger organizations still tend to maintain more diverse projects, attract broader developer attention, and connect multiple technical subfields, which gives them higher centrality in the sampled AI repository ecosystem.

9 Visualization Protocol

9.1 Why "hairball" happens

Dense cross-community edges and high edge-count per node can collapse community separation under force-directed layouts. To improve readability, we generated a clustered variant (`company_network_clustered.gexf`) by preserving strong intra-community links and only a small number of bridge edges.

9.2 Recommended Gephi Layout Template

- Layout: ForceAtlas2
- LinLog mode: ON
- Prevent overlap: ON
- Gravity: 0.05
- Scaling: 180
- Edge weight influence: 3.5
- Dissuade hubs: ON
- Then run Noverlap

Color nodes by `CompanyCommunity`, size nodes by `RepoCount` or `PageRank`.

10 Threats to Validity

- **Company identity noise**: profile `company` fields are user-provided and can be missing, inconsistent, or stylistically heterogeneous.
- **Sampling bias**: top-star sampling over-represents popular repositories.
- **Metadata incompleteness**: README and dependency files are not uniformly available.
- **Method sensitivity**: community assignments depend on backbone and resolution parameters.
- **Temporal bias**: single snapshot may hide dynamics of ecosystem evolution.

11 Reproducibility

11.1 Environment

Conda environment: `cours`. Required libraries include: `requests`, `networkx`, `pandas`, `python-louvain`.

11.2 Main Commands

Stage A (initial repository-level run, 100+100)

```
python scripts/build_repository_backbone.py \
  --reuse-raw \
  --raw-output data/raw/repo_raw_data_fork_top100.json \
  --gexf-output outputs/graphs/repo_signal_compare/repository_mixed_weight.gexf \
  --signal-mode mixed \
  --readme-similarity-version v2 \
  --weight-fork 0.6 --weight-readme 0.2 --weight-dep 0.2 \
  --seed-community-layout

python scripts/build_repository_backbone.py \
  --reuse-raw \
  --raw-output data/raw/repo_raw_data_fork_top100.json \
  --gexf-output outputs/graphs/repo_signal_compare/repository_fork_only.gexf \
  --signal-mode fork \
  --pre-min-weight 0.0125 --min-weight 0.0125 \
  --top-k 0 --k-core 0 --no-auto-relax \
  --layout-reference-gexf outputs/graphs/repo_signal_compare/repository_mixed_weight.gexf

python scripts/build_repository_backbone.py \
  --reuse-raw \
  --raw-output data/raw/repo_raw_data_fork_top100.json \
  --gexf-output outputs/graphs/repo_signal_compare/repository_readme_only.gexf \
  --signal-mode readme \
  --readme-similarity-version v2 \
  --pre-min-weight 0.07 --min-weight 0.07 \
  --top-k 0 --k-core 0 --no-auto-relax \
  --layout-reference-gexf outputs/graphs/repo_signal_compare/repository_mixed_weight.gexf
```

Stage B (expanded run for company analysis, 500+500)

```
python scripts/build_repository_backbone.py \
  --raw-output data/raw/repo_raw_data_fork.json \
  --gexf-output outputs/graphs/repo/repository_backbone_fork.gexf \
  --per-domain-limit 500 \
  --weight-fork 0.6 --weight-readme 0.2 --weight-dep 0.2
```

Company aggregation (used in Stage B)

```
python scripts/analyze_company_communities.py \
  --repo-raw data/raw/repo_raw_data_fork.json \
  --repo-graph outputs/graphs/repo/repository_backbone_fork.gexf \
  --company-graph outputs/graphs/company/company_network.gexf \
  --company-nodes-csv outputs/tables/company_nodes.csv \
  --company-communities-csv outputs/tables/company_communities.csv \
  --owner-company-map-csv outputs/tables/owner_company_map.csv \
  --fetch-owner-company \
  --drop-owners-without-company
```

12 Conclusion

We implemented and validated an end-to-end, scalable graph-analysis pipeline from repository retrieval to company-level community aggregation. The current results support a structured ecosystem with multiple large ownership communities and high prevalence of small entities. Next steps should focus on (i) stronger owner-to-company normalization, (ii) parameter robustness experiments, and (iii) temporal analysis for community evolution.